

Algoritmi e Protocolli per la Sicurezza

Project Work:

Progettazione di un Sistema di Voto Elettronico per Elezioni Universitarie

Raimondo Domenico Sambuco Salvatore
Matr. **IE22700284** Matr. **IE22700305**

1 WP1 - Modello

Il sistema deve gestire un'elezione universitaria di tutti gli Organi di Ateneo. Il contesto di questo specifico progetto sarà l'Università degli Studi di Salerno, in particolare di un'elezione per il rinnovo del Consiglio degli Studenti. Le considerazioni e gli sviluppi dedicati a questa particolare applicazione saranno tali da poter essere estesi a casi più generali, quali: Ateneo diverso da quello di Salerno, elezioni più generiche rispetto a quelle relative al solo Consiglio degli Studenti.

1.1 Considerazioni sulla scelta del tipo di elezione

Sono state analizzate le diverse possibilità sulla tipologia di voto, con il relativo impatto sulla progettazione e le relative implicazioni di sicurezza.

- **Referendum binario:** Il voto si esprime scegliendo tra due alternative. Rappresenta la soluzione più semplice, favorendo notevolmente la semplicità crittografica e l'efficienza, grazie al fatto che il voto viene memorizzato come un singolo bit. Ha, di contro, lo svantaggio di essere poco applicabile ad un contesto universitario, in cui vi sono diversi organi da eleggere, ognuno dei quali con svariati candidati;
- **Elezione con lista chiusa:** Permette maggiore flessibilità rispetto alla soluzione precedente, rispettando il realismo di un'elezione universitaria senza complicare eccessivamente il controllo crittografico. Di contro, non è la massima espressione della libertà di voto tipicamente offerta nelle reali elezioni accademiche, nelle quali solitamente è possibile selezionare i singoli candidati. Rispetto al Referendum Binario, aumenta il rischio di manipolazione della Lista;
- **Elezione con preferenza nominale:** La soluzione migliore in termini di realismo, rappresentando a pieno il contesto universitario reale. È, in maniera linearmente conseguente, l'alternativa col più alto fattore di difficoltà contemporaneamente sul piano implementativo e di semplicità crittografica.

In conseguenza alle considerazioni effettuate, la nostra scelta è ricaduta sul voto basato su *elezione con lista chiusa*, poiché rappresenta il compromesso più bilanciato tra realismo e realizzabilità in termini di sicurezza.

1.2 Definizione degli Attori Onesti

Definiamo quattro attori principali all'interno dell'architettura:

- **Elettore**: Si autentica tramite il sistema di Ateneo, seleziona la lista, invia il voto cifrato e, a valle del processo, verifica l'effettiva acquisizione della propria preferenza. L'elettorato è definito **per organo**: per il Consiglio degli Studenti (caso di studio del progetto) votano i **soli studenti regolarmente iscritti**; i docenti rientrano nell'elettorato di altri organi (es. Senato Accademico) supportati in modo modulare dal sistema. Questa distinzione sarà realizzata tramite il campo di categoria del token (`eligible_scope`, Sezione 2.2).
- **Servizio di Autenticazione (AS)**: L'entità che gestisce l'accesso e l'identità civile. Il suo compito è controllare che chi ha effettuato il login sia avente diritto al voto e rilasciare un'autorizzazione anonima al client. Nel nostro contesto, il servizio di Autenticazione è costituito dalla piattaforma di Ateneo stessa, la quale, a sua volta, fa affidamento a servizi di autenticazione certificati a livello nazionale come SPID e CIE.
- **Autorità Elettorale (EA)**: Gestisce il sistema di voto agendo come un'urna digitale. Riceve la conferma anonima del diritto al voto, accetta la scheda cifrata e la memorizza in maniera del tutto slegata dall'identità fisica dell'utente per garantirne la segretezza assoluta. A elezioni concluse, è l'entità preposta all'apertura dell'urna e allo scrutinio dei voti.
- **Bacheca Pubblica (Public Bulletin Board - PBB)**: Un'entità logica indipendente strutturata come registro di sola aggiunta. Il suo ruolo è ricevere i dati dall'Autorità Elettorale e renderli universalmente accessibili. Il suo scopo primario non è la riservatezza, bensì l'integrità e la trasparenza: espone i risultati aggregati in chiaro e l'elenco delle ricevute crittografiche, mitigando eventuali abusi di fiducia da parte dei gestori di sistema.

1.3 Definizione del Threat Model

La definizione di un Threat Model ci permette di avere sempre chiare le minacce a cui il sistema va incontro, permettendo di conseguenza di lavorare per eliminare o mitigare ogni possibile attacco alla sicurezza. Nel contesto della progettazione del sistema di voto elettronico, assumeremo tutti gli attaccanti come avversari PPT (Tempo Polinomiale Probabilistico), il che significa che non hanno potenza computazionale illimitata, ma sono comunque in grado di eseguire attacchi sofisticati entro limiti ragionevoli di tempo e risorse. In particolare, distinguiamo quattro macro-categorie di minacce, ognuna con i suoi attori specifici:

- **Insider Threats**:
 - **Alberto l'Einsteinio**
 - * **Tipologia**: Attacco all'unicità, **Attivo**
 - * **Descrizione**: Elettore che prova a votare due volte per la ragazza che gli piace.
 - * **Risorse**: L'avversario agisce possedendo credenziali legittime di base (accesso standard autorizzato dal Servizio di Autenticazione), ma è privo di privilegi amministrativi sul sistema.

– **Gianfranco il traditore**

- * **Tipologia:** Attacco all'integrità, [Attivo](#)
- * **Descrizione:** Entrato come curatore del server in cui sono conservati i voti, ha intenzione di modificarli e cancellarli per favorire l'elezione di chi l'ha assunto segretamente.
- * **Risorse:** Avversario con privilegi di amministrazione. Possiede accesso diretto al database dell'Autorità Elettorale e alla memoria del server, ma è limitato computazionalmente (non possiede potenza sufficiente per invertire funzioni di Hash o forzare la crittografia asimmetrica).

– **Evil Unisa**

- * **Tipologia:** Attacco all'integrità istituzionale e alla segretezza, [Attivo](#)
- * **Descrizione:** Amministratore corrotto o server dell'Ateneo (AS) violato che manipola il database anagrafico per generare token autorizzativi fraudolenti, legittimamente firmati ma associati a identità sintetiche o a soggetti non aventi diritto, alterando il quorum. Inoltre, controllando i contenuti serviti dal portale di Ateneo, può tentare di **sostituire materiale crittografico distribuito ai client** (in particolare chiavi pubbliche), qualora la distribuzione delle chiavi transitasse dal suo dominio.
- * **Risorse:** Avversario con massimi privilegi sul dominio d'identità universitario. Detiene il controllo della chiave privata di firma dell'Ateneo (SK_{AS}) e dei contenuti web serviti dal portale, potendo ingannare l'Electoral Authority esterna, la quale convaliderà i permessi come autentici. Non controlla il dominio dell'EA né la catena di certificazione X.509 pubblica.

– **Asse Learning-Election**

- * **Tipologia:** Attacco alla riservatezza / Violazione dello Pseudoanonimato, [Attivo](#)
- * **Descrizione:** Cooperazione illecita in cui gli amministratori del Servizio di Autenticazione (AS) e dell'Autorità Elettorale (EA) incrociano i rispettivi log temporali e di rete per ricollegare l'identità civile della matricola alla specifica busta crittografata depositata nell'urna.
- * **Risorse:** Coalizione di avversari interni al sistema. Possiedono pieni poteri di lettura e accoppiamento sulle tabelle relazionali e sui timestamp di sessione memorizzati sui due server indipendenti.

• **Furto di Identità e Phishing:**

– **Malcolm in the Middle**

- * **Tipologia:** Attacco di Intercettazione, [Ibrido](#)
- * **Descrizione:** Si mette in ascolto sul canale di comunicazione tra Servizio di Autenticazione e Autorità Elettorale, rubando l'identità di un Elettore idoneo al voto perché suo zio si è candidato e vuole votarlo.
- * **Risorse:** Possiede il controllo completo del canale di comunicazione (capacità di intercettazione, alterazione e iniezione di messaggi in transito), ma non ha compromesso fisicamente gli endpoint (client dello studente o server d'Ateneo).

– **Votazioni.UMISA.IT**

- * **Tipologia:** Phishing / Ingegneria sociale, **Attivo**
- * **Descrizione:** Allestisce un sito civetta dal nome simile a quello istituzionale e tenta di **reindirizzare attivamente** l'elettore su sé stesso invece che sul sito corretto (link esca, e-mail contraffatte, typosquatting), così da ottenere dati sensibili. Pur non manomettendo il canale di rete, a differenza di *Malcolm*, l'avversario manipola il comportamento dell'utente: l'attacco è quindi di natura attiva.
- * **Risorse:** Avversario capace di allestire infrastrutture di rete malevole esterne. Non possiede accesso all'infrastruttura d'Ateneo, ma tenta di sfruttare il fattore umano trasformando l'utente in un oracolo per possibili attacchi a testo in chiaro scelto.

• **Denial of Service e Hacktivism:**

– **Gruppo Blackwatch**

- * **Tipologia:** Attacco alla disponibilità, **Attivo**
- * **Descrizione:** Attacca l'Autorità Elettorale per mandarla fuoriuso e impedire a tutti gli elettori di poter votare.
- * **Risorse:** Avversario dotato di elevate risorse di rete distribuite per attacchi volumetrici o a livello applicativo. Sebbene disponga di un'alta capacità di invio richieste per saturare la banda, è vincolato da limiti computazionali che gli impediscono di rompere le primitive crittografiche del sistema.

• **Supply-Chain Attacks:**

– **Alessandria Project**

- * **Tipologia:** Attacco all'integrità, **Attivo**
- * **Descrizione:** Libreria open-source che, stranamente, offre funzioni un po' troppo utili. È in realtà compromessa e, una volta che il sistema è funzionante, altera il voto dell'elettore un attimo prima di inviarlo.
- * **Risorse:** Avversario con capacità di esecuzione di codice arbitrario in locale. Sfruttando la fiducia nella supply chain, agisce all'interno dello spazio di memoria dell'applicazione client o server, avendo potenziale accesso ai dati in chiaro prima dell'incapsulamento crittografico.

1.4 Valutazione delle proprietà da preservare

Il sistema dovrà assicurare la **segretezza** del voto, garantendo **confidenzialità**; contemporaneamente sarà fondamentale concentrarsi sull'**unicità** del voto, così da evitare possibili manipolazioni da parte di elettori che tentano di votare più volte. Viene tenuto presente che la segretezza e l'unicità sono proprietà che possono tra loro essere in conflitto, poiché per garantire l'unicità è necessario che il sistema sia in grado di identificare l'elettore, mentre per garantire la segretezza è necessario che il voto sia slegato dall'identità di chi ha espresso il voto. Per questo motivo, è fondamentale progettare un sistema che riesca a bilanciare queste due proprietà in maniera efficace. Il sistema in esame prevederà la non **revocabilità** e la non **sostituzione** del voto, evitando dunque che un elettore possa modificare o cancellare il proprio voto dopo averlo espresso. Sarà inoltre importante garantire l'**integrità** del voto, così da assicurare che il voto espresso dall'elettore sia quello effettivamente memorizzato e conteggiato dal sistema. Ciò implicherà la capacità dell'autorità elettorale di verificare che il voto ricevuto non sia stato alterato durante la trasmissione o la memorizzazione.

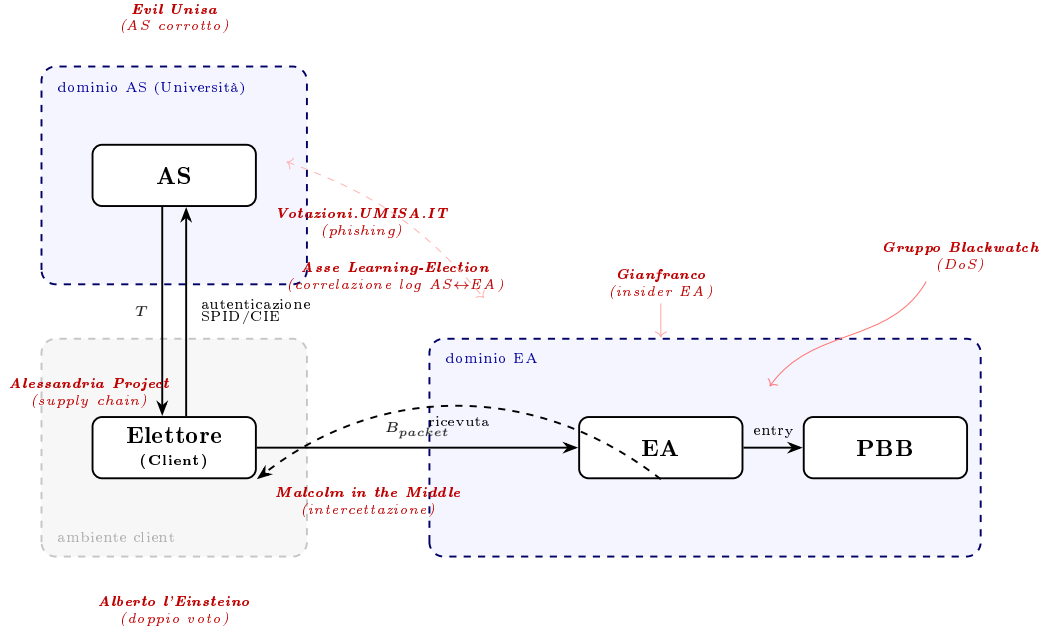


Figura 1: Data Flow Diagram del Threat Model: componenti onesti (riquadri) nei loro domini di fiducia (tratteggiato blu), flussi di dati principali e avversari (*rosso*) posizionati nel loro dominio di azione.

In contrasto alla possibilità di un attacco di Denial of Service, sarà necessario garantire la **disponibilità** (availability) del sistema, così da permettere a tutti gli elettori di poter esprimere il proprio voto senza interruzioni o impedimenti.

Nel contesto del nostro progetto, è importante sottolineare la fiducia che si ripone nel sistema di autenticazione di Ateneo, il quale permette di garantire l'**autenticità** degli elettori. In tal modo, è possibile assicurare che solo gli elettori autorizzati possano accedere al sistema di voto e esprimere la propria preferenza, contribuendo così a mantenere l'integrità e la legittimità dell'elezione.

Inoltre, l'autenticazione di Ateneo stessa sfrutta sistemi di autenticazione certificati a livello nazionale come SPID e CIE, che prevedono uno scambio iniziale di informazioni 'out-of-band' come, nel nostro caso, uffici di segreteria che verificano l'identità dell'elettore e gli forniscono le credenziali per accedere al sistema di autenticazione. Questo processo contribuisce a rafforzare ulteriormente la sicurezza del sistema, poiché rende più difficile per un attaccante compromettere l'autenticazione senza avere accesso fisico alle credenziali dell'utente.

In aggiunta alle proprietà classiche della sicurezza (Riservatezza, Integrità, Disponibilità), il sistema di voto elettronico dovrà soddisfare come requisito primario la **Verificabilità** (*Verifiability*), che si declina in due proprietà complementari e irrinunciabili:

- **Verificabilità Individuale:** Ogni elettore deve avere la garanzia matematica di poter controllare che la propria scheda sia stata correttamente depositata e conteggiata dal sistema. Questo controllo avverrà tramite la Bacheca Pubblica, fornendo all'utente una "ricevuta crittografica" che dimostri la presa in carico senza mai violare la segretezza della preferenza.
- **Verificabilità Universale:** Chiunque (candidati, elettori o auditor esterni) deve poter accedere ai dati della Bacheca Pubblica per accertarsi che lo scrutinio finale sia l'esatto e inalterato risultato dei voti validi registrati.

Infine, per quanto concerne il rischio di **Coercizione**, occorre fare una precisazione. Sebbene in ambito crittografico esistano soluzioni avanzate per mitigare la coercizione, l'implementazione di tali schemi complicherebbe esponenzialmente il protocollo e l'esperienza utente, violando i vincoli di efficienza. Pertanto, nel presente progetto, si accetta il rischio residuo di coercizione fisica limitata all'istante dell'espressione del voto: tale rischio è tipico di qualunque sistema di voto elettronico remoto in ambiente non controllato e non è eliminabile con le sole primitive crittografiche studiate nel corso. Si mitiga parzialmente l'impatto della compravendita su larga scala imponendo l'irrevocabilità della scheda — un attaccante non potrà usare credenziali rubate per sovrascrivere un voto già espresso — ma questa misura non impedisce la coercizione nell'istante del voto e va intesa come mitigazione di scenari specifici, non come difesa completa.

Consapevoli del fatto che vi sono minacce a fronte delle quali non esistono soluzioni nell'ambito della teoria dell'informazione e contemporaneamente considerando che non sempre la conoscenza dell'attaccante è proporzionata alla potenza computazionale (come nel caso di attacchi *brute force*), proseguiamo lo sviluppo del progetto concentrando la nostra attenzione sulle proprietà di sicurezza che è possibile garantire, cercando di mitigare o eliminare le minacce identificate nel nostro Threat Model.

1.5 Confini di Fiducia e Rischio Residuo

Questa sezione non anticipa la soluzione, ma si limita a dichiarare in modo esplicito le assunzioni di fiducia su cui il modello poggia e ad analizzarne il rischio residuo, come richiesto dal requisito di trasparenza. La realizzazione tecnica di tali confini (meccanismi di autenticazione, distribuzione delle chiavi, separazione dei domini) è rimandata interamente al WP2. Il bilanciamento tra pseudoanonimato e tracciabilità del diritto al voto richiede di definire con rigore i confini di fiducia (*Trust Boundaries*) tra le entità del protocollo:

- **Assunzione sull'emissione delle identità:** Si assume che il diritto di voto sia ancorato a un *Identity Provider* nazionale certificato (SPID e CIE), rispetto al quale l'Ateneo agisce come semplice verificatore. Sotto questa assunzione, l'Ateneo non ha il potere computazionale né giuridico di falsificare identità digitali a livello centrale, per cui la minaccia di generazione di elettori fittizi (*identità sintetiche*) si riduce alla sola correttezza del database degli iscritti regolari. *Rischio residuo:* un AS interamente compromesso (minaccia *Evil Unisa*) potrebbe comunque emettere token per soggetti non aventi diritto attingendo al proprio bacino anagrafico; questa eventualità è affidata agli audit interni dell'Ateneo.
- **Assunzione di separazione di fiducia:** Si assume che il dominio del Servizio di Autenticazione (AS) e quello dell'Autorità Elettorale (EA) siano amministrativamente e infrastrutturalmente indipendenti, senza canali di comunicazione diretti server-to-server. Sotto questa assunzione, de-anonimizzare un elettore richiede di compromettere contemporaneamente due perimetri distinti, rendendo intrattabile la collusione (minaccia *Asse Learning-Election*). *Rischio residuo:* una collusione intenzionale, sincrona e coordinata tra le due organizzazioni esula dalle barriere puramente crittografiche e viene affidata alle tutele legali post-elettorali.
- **Assunzione sulla distribuzione delle chiavi:** Si assume che il materiale crittografico pubblico necessario al client (in particolare la chiave di cifratura dell'urna) sia verificabile attraverso **canali indipendenti dall'AS:** nessuna entità il cui dominio è già contemplato come potenzialmente ostile nel Threat Model (*Evil Unisa*) deve trovarsi da sola sul percorso di distribuzione delle chiavi. La realizzazione tecnica di questo vincolo (certificati X.509, fingerprint pubblicato su PBB e su canale istituzionale out-of-band) è demandata al WP2. *Rischio residuo:* la compromissione simultanea della catena di certificazione pubblica e dei canali di pubblicazione indipendenti, considerata fuori scopo per un avversario PPT con le risorse dichiarate.

2 WP2 - Descrizione della Soluzione Progettuale

La seguente sezione delinea la trasposizione architetturale del protocollo di voto elettronico per il rinnovo del Consiglio degli Studenti dell'Università degli Studi di Salerno. Coerentemente con le scelte strategiche e i vincoli di efficienza computazionale prefissati in fase di modellazione, il sistema è basato sull'impiego di Token JSON firmati digitalmente e su un controllo d'accesso basato sui ruoli (RBAC).

Questa architettura realizza un disaccoppiamento completo tra l'identità anagrafica dell'elettore e la preferenza espressa, minimizzando le assunzioni di fiducia e strutturando i dati in modo da consentire una rigorosa analisi di sicurezza nel successivo WP3.

2.1 Tipo di Voto e Contesto Applicativo

Come definito nel modello dei requisiti, il sistema è configurato per gestire un'elezione a **lista chiusa**. Questa scelta permette di strutturare la scheda elettorale come un identificativo stringa compatto (associato alla lista selezionata), semplificando la logica di validazione in fase di scrutinio e riducendo il rischio di errori nell'analisi del plaintext decifrato. La scelta garantisce il bilanciamento tra il realismo del contesto accademico e la sostenibilità delle risorse computazionali del client.

Confronto con le alternative. Le tre tipologie di voto producono strutture di plaintext sostanzialmente diverse, con implicazioni dirette sulla complessità dello scrutinio. A titolo illustrativo:

```
1 { "voto": 1, "salt_nonce": "a5f8e39b" }
```

Listing 1: Referendum binario

```
1 { "elezione_id": "UNISA-CONS-STUD-2026",  
2   "scelta_lista": "LISTA_02",  
3   "salt_nonce": "a5f8e39b2c1d9f4e" }
```

Listing 2: Lista chiusa (soluzione adottata)

```
1 { "elezione_id": "UNISA-CONS-STUD-2026",  
2   "candidati": ["Mario Rossi", "Anna Bianchi"],  
3   "salt_nonce": "a5f8e39b2c1d9f4e" }
```

Listing 3: Preferenza nominale

Una precisazione crittografica rilevante: con RSA-OAEP il testo cifrato ha lunghezza **sempre fissa** (512 byte per una chiave da 4096 bit), indipendentemente dalla lunghezza del plaintext. La differenza tra le tre tipologie non riguarda quindi la dimensione del ciphertext, che è invariante, bensì la **complessità della logica di scrutinio**: il referendum binario richiede solo di verificare che il valore decodificato appartenga a $\{0, 1\}$; la lista chiusa richiede un confronto con un insieme fisso di stringhe; la preferenza nominale impone la validazione di un array a cardinalità variabile, il controllo dei nomi dei candidati, l'assenza di duplicati e il rispetto dei limiti di selezione. È quest'ultima complessità, non la dimensione del ciphertext, a motivare la scelta della lista chiusa rispetto alla preferenza nominale.

2.2 Fase di Registrazione e Autenticazione

Il protocollo si basa sulla separazione netta dei compiti (*Separation of Duty*) tra due entità indipendenti che non condividono alcuna base di dati comune: il **Server di Autenticazione (AS)** e l'**Autorità Elettorale (EA)**.

L'elettore avvia la procedura interfacciandosi con il portale dell'infrastruttura di Ateneo.

1. L'elettore esegue il login primario autenticandosi tramite i canali nazionali certificati SPID o CIE integrati nei sistemi di Ateneo.
2. L'AS interroga la base dati interna degli studenti regolarmente iscritti per verificare il possesso del diritto di voto attivo per lo specifico Consiglio degli Studenti.
3. **Meccanismo Anti-Login Duplicato e Derivazione del Token:** All'atto della verifica positiva, l'AS imposta il flag `token_issued[(identity_id, election_id)] = True`. Il `token_id` non è generato casualmente, ma **derivato deterministicamente** tramite funzione pseudocasuale con chiave (PRF):

$$\text{token_id} = \text{HMAC-SHA256}(K_{\text{HMAC}}, \text{identity_id} \parallel \text{election_id})$$

dove K_{HMAC} è una chiave dedicata, **distinta dalla chiave di firma SK_{AS}** (principio di separazione delle chiavi per scopo). Grazie a questa derivazione, su qualsiasi re-autenticazione (inclusi i casi di crash e perdita del token), l'AS può ricalcolare lo stesso `token_id` e ri-emettere il token identico **senza memorizzare alcuna mappatura**. La protezione contro il doppio voto non dipende più dal flag come gate assoluto, ma dall'unicità del `token_id` lato EA. Il flag mantiene comunque una funzione di controllo indipendente: se il Registro dei Token Spesi dell'EA fosse compromesso, il flag impedirebbe all'AS di emettere token aggiuntivi per lo stesso elettore nella stessa elezione, fornendo un secondo livello di protezione autonomo. Il campo `expires_at` coincide con l'**orario di chiusura delle urne**, comune a tutti i token dell'elezione: re-emissioni successive alla chiusura sono quindi tecnicamente possibili ma prive di effetto, poiché l'EA non accetta più ballot.

Custodia di K_{HMAC} (requisito strutturale, non opzionale). Chiunque conosca K_{HMAC} può ricalcolare il `token_id` di ogni elettore e, confrontandolo con la PBB pubblica, collegare l'identità alla scheda cifrata. Per questo K_{HMAC} è trattata con le **stesse garanzie di custodia di una chiave privata**: viene generata in una cerimonia delle chiavi precedente all'indizione, caricata in un **modulo hardware di sicurezza (HSM) in modalità non esportabile** (l'HSM espone la sola operazione di HMAC, mai i byte della chiave) e il suo backup è suddiviso tramite **condivisione del segreto di Shamir** (t, n) tra i membri di una commissione di garanzia indipendente dall'amministrazione sistemistica dell'AS. Ogni invocazione dell'HSM è registrata su un **log ad accodamento firmato**: un'eventuale campagna di ricalcolo massivo dei `token_id` (necessaria per la de-anonimizzazione) lascerebbe una traccia di volume anomalo verificabile in sede di audit. In questo modo la compromissione del *solo software* dell'AS non è più sufficiente a ottenere la mappatura identità $\leftrightarrow$ token_id: serve la violazione del perimetro hardware o il raggiungimento del quorum t della commissione di garanzia.

4. **Generazione del Payload Anonimo:** L'AS istanzia un payload in formato JSON strutturato in modo da non contenere alcun riferimento (matricola, codice fiscale, nome) che possa ricondurre all'identità reale dell'elettore:

```
1 {  
2     "token_id": "<HMAC-SHA256(K_HMAC, identity_id ||  
3         election_id)>",  
4     "election_id": "UNISA-CONS-STUD-2026",  
5     "eligible_scope": "STUDENT",  
6     "expires_at": "2026-05-25T15:00:00Z"  
}
```

Listing 4: Struttura del Token JSON di Voto

Il campo `identity_id` è costituito dall'identificativo di ateneo, con la matricola come miglior candidato per gli studenti e il "codice docente" per gli insegnanti. Il token non contiene alcun campo `role`: l'unico discriminante per l'accettazione o il rifiuto di un ballot è lo stato del `token_id` nel Registro dei Token Spesi dell'EA. Il campo `election_id` lega il token a una specifica elezione; il campo `eligible_scope` indica la categoria dell'elettore (STUDENT o PROFESSOR), necessaria per filtrare la partecipazione agli organi esclusivi di ciascuna categoria. Per il Consiglio degli Studenti il valore sarà sempre STUDENT, ma il campo supporta in modo modulare gli altri organi di Ateneo (es. Senato Accademico), coerentemente con la vocazione estensibile dichiarata in Sezione 1.1.

5. **Sigillatura Crittografica:** Il payload viene serializzato e **firmato** dal Server di Autenticazione secondo il paradigma *hash-and-sign*: se ne calcola l'impronta tramite SHA-256 e si applica la firma con la chiave privata (SK_{AS}). Il Token risultante

$$T = \text{Token} \parallel \text{Firma}_{SK_{AS}}$$

viene trasmesso al browser del client su canale cifrato TLS.

Confronto con le alternative. Il meccanismo adottato realizza il modello di *identità federata* studiato nel corso, in cui l'AS funge da *Relying Party* verso un *Identity Provider* nazionale. Rispetto a un'autenticazione *password-based* gestita localmente dall'Ateneo, questa scelta elimina la custodia di un database di segreti condivisi (e quindi l'esposizione ad attacchi di *password spraying*, *credential stuffing*, dizionario e forza bruta sul fronte del voto) e demanda l'*identity proofing* a un'infrastruttura che già lo esegue out-of-band con garanzie di legge. Rispetto a un modello di identità puramente *centralizzato*, riduce il numero di entità che custodiscono dati personali; rispetto a soluzioni *passwordless* come FIDO2, evita l'onere di registrare e gestire autenticator hardware per l'intera popolazione studentesca, pur restandone una possibile estensione futura per innalzare il livello di garanzia.

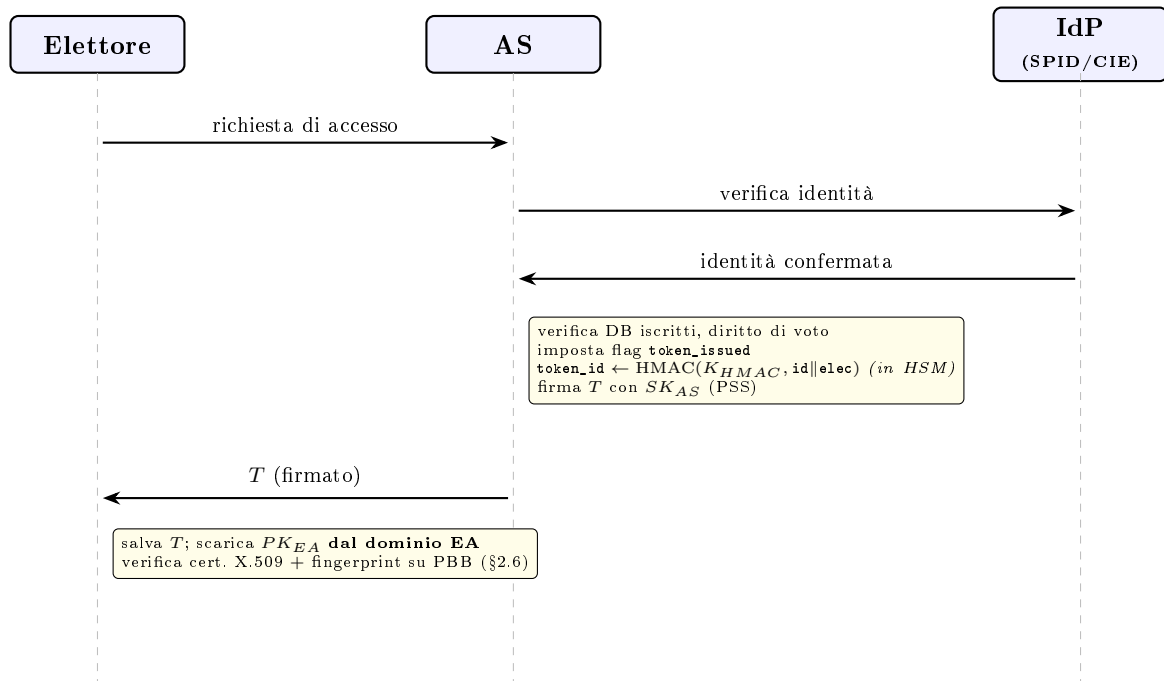


Figura 2: Diagramma di sequenza della fase di autenticazione (§2.2).

2.3 Pseudoanonimato del Voto e Separazione di Fiducia

Il vincolo supremo dettato dal modello impone che nessuna entità singola sia in grado di associare l'identità civile alla preferenza di voto. Il protocollo realizza questo principio tramite un disaccoppiamento architetturale asimmetrico in logica federata:

- L'**AS** detiene esclusivamente la conoscenza del "*Chi ha effettuato l'accesso*" e non interagirà mai, né direttamente né indirettamente, con la scheda elettorale cifrata, non possedendo le chiavi per decifrare l'urna.
- L'**EA** detiene esclusivamente la conoscenza del "*Cosa è stato votato*". Non avendo accesso alle tabelle anagrafiche degli studenti o ai log di autenticazione dell'AS, l'EA vede unicamente passare un identificativo esadecimale pseudorandomizzato (`token_id`) privo di legami biometrici o nominali.

Di conseguenza, il Rischio Residuo di tracciamento o de-anonimizzazione dell'elettore è confinato a due scenari, entrambi dichiarati. **(a) Collusione AS+EA** (*Asse Learning-Election*): richiede la compromissione coordinata di due perimetri amministrativi indipendenti ed è affidata alle tutele legali post-elettorali. **(b) Compromissione della derivazione deterministica**: va dichiarato senza eufemismi che, se K_{HMAC} fosse ottenibile, il suo detentore potrebbe ricalcolare il `token_id` di *qualsiasi* elettore e confrontarlo con i token pubblicati nella PBB, collegando **da solo e senza correlazione temporale** l'identità civile alla scheda cifrata, un degrado qualitativo rispetto al design a `token_id` casuale, in cui tale collegamento richiedeva la collusione di entrambi i domini. È esattamente per questo che la custodia di K_{HMAC} in HSM non esportabile, con backup a soglia di Shamir e log firmato delle invocazioni (Sezione 2.2), non è una raccomandazione accessoria ma un **requisito strutturale del protocollo**: sotto tale custodia, la compromissione del solo software dell'AS non restituisce la mappatura, e il rischio (b) si riduce alla violazione del perimetro hardware o al raggiungimento fraudolento del quorum t della commissione di garanzia. Il vantaggio progettuale della derivazione (il recupero del token dopo crash e l'eliminazione del problema di re-emissione sicura) è ritenuto proporzionato al rischio così mitigato.

Politica di logging. L'assunzione secondo cui l'AS non conserva il `token_id` deve estendersi anche alla **politica dei log applicativi**: il server di autenticazione deve evitare di registrare il payload completo del token nei log HTTP, nei log di applicazione e nei sistemi di monitoraggio. Sono ammessi soltanto log minimali e non collegabili, ad esempio la sola registrazione dell'avvenuta emissione di un token per un utente avente diritto, senza memorizzare il valore del `token_id` né identificativi tecnici riutilizzabili dall'EA per ricostruire l'associazione identità↔scheda.

2.4 Formato dei Messaggi e delle Schede di Voto Cifrate

Il flusso informativo prevede scambi strutturati per garantire l'immutabilità dei dati d'urna.

2.4.1 La Scheda Elettorale in Chiaro (M)

Prima di subire i processi di offuscamento crittografico, la preferenza espressa dal client viene codificata in un blocco JSON compatto, inclusivo di un nonce di randomizzazione per impedire attacchi basati su dizionario (visto il numero limitato di liste candidate):

```
1 {  
2     "elezione_id": "UNISA-CONS-STUD-2026",  
3     "scelta_lista": "LISTA_02",  
4     "salt_nonce": "a5f8e39b2c1d9f4e"  
5 }
```

Listing 5: Struttura della Scheda in Chiaro

Il campo `scelta_lista` è popolato dal client a partire dal sottoinsieme finito e pubblicamente noto delle liste in competizione per l'elezione identificata da `elezione_id`. È tuttavia importante dichiarare un limite intrinseco: poiché la scheda viene cifrata sul dispositivo dell'elettore, **il protocollo non può garantire a priori che il valore cifrato corrisponda a una lista valida**. Un elettore malevolo (o un client compromesso, cfr. minaccia *Alessandria Project*), pur in possesso di un token legittimo, può confezionare un B_{packet} il cui testo cifrato decifra in un identificativo inesistente o malformato. Tale eventualità non costituisce una falla critica (non altera né rivela il voto altrui) ma va gestita esplicitamente: la **validazione di buona formazione** della scheda (coerenza dell'`elezione_id` e appartenenza di `scelta_lista` all'insieme ammesso) viene eseguita dall'EA *dopo* la decifrazione, in fase di scrutinio (cfr. Sezione 2.8); le schede non conformi vengono classificate come **nulle** ed escluse dal conteggio.

Per preservare la trasparenza e consentire la **verifica individuale della validità del proprio voto**, l'EA pubblica sulla PBB due informazioni distinte: l'elenco dei voti in chiaro validi (rimiscolati) e l'**elenco dei leaf_hash delle schede nulle**. Ogni elettore può confrontare il proprio `leaf_hash` (dalla ricevuta) con quest'ultima lista per accertare se il proprio voto è stato conteggiato come valido o scartato, senza che ciò riveli il contenuto del voto. Va dichiarato il tradeoff residuo: l'assenza del proprio `leaf_hash` dalla lista dei nulli informa un eventuale coercitore che l'elettore ha espresso una preferenza valida, pur senza rivelarne il contenuto.

2.4.2 Il Pacchetto di Sottomissione del Voto (B_{packet})

Il client esegue la cifratura del blocco JSON della scheda utilizzando la chiave pubblica dell'Autorità Elettorale (PK_{EA}) ottenendo il testo cifrato $C = E_{PK_{EA}}(M)$. Il messaggio finale trasmesso all'EA si compone dei seguenti campi strutturati:

$$B_{packet} = \{C \parallel T\}$$

L'assenza di firme apposte con chiavi private generate localmente dal client elimina qualsiasi traccia crittografica che possa ricondurre il pacchetto all'host mittente, blindando lo pseudoanonimato a livello di rete.

Confronto con le alternative. La letteratura sul *signcryption* studiata nel corso suggerirebbe di accoppiare cifratura e firma (*encrypt-then-authenticate* o *authenticate-then-encrypt*) per ottenere congiuntamente segretezza e integrità del mittente. In questo contesto, però, una firma generata dal client sarebbe controproducente: la sua **trasferibilità** e la conseguente proprietà di **non ripudio** consentirebbero all'elettore di dimostrare a terzi come ha votato, indebolendo proprio la segretezza e la resistenza alla coercizione che il sistema vuole tutelare. Per questo l'autenticità del diritto al voto non viene affidata a una firma del client, bensì al token T già firmato dall'AS e incapsulato nel pacchetto: l'integrità e l'origine dell'autorizzazione restano verificabili (tramite PK_{AS}), mentre la preferenza in chiaro M resta protetta dalla sola cifratura asimmetrica verso l'EA, senza alcun legame firmabile con l'identità del votante.

2.5 Operazioni Crittografiche

Per ottemperare ai requisiti di efficienza computazionale e sicurezza, vengono impiegate esclusivamente primitive crittografiche standardizzate e consolidate:

1. **Cifratura Asimmetrica (RSA-OAEP):** La scheda di voto in chiaro M viene cifrata dal client tramite algoritmo RSA combinato con lo schema di padding ottimale OAEP (*Optimal Asymmetric Encryption Padding*). Questo schema introduce una componente stocastica che rende la cifratura semanticamente sicura e resistente alla manipolazione dei testi cifrati (CCA). La lunghezza della chiave scelta è pari a 4096 bit.

2. **Firme Digitali (*hash-and-sign*):** Token e ricevute sono autenticati seguendo il paradigma *hash-and-sign* studiato nel corso (RSA-FDH): si calcola l'impronta SHA-256 del messaggio e la si firma con la chiave privata del firmatario, mentre chiunque disponga della corrispondente chiave pubblica può verificarla. L'applicazione della funzione di hash prima della firma è ciò che neutralizza gli attacchi alla firma RSA grezza (*no-message attack* e falsificazioni che sfruttano la proprietà omomorfa). In sede implementativa (WP4) la firma sarà istanziata con lo schema probabilistico **RSA-PSS**, che irrobustisce ulteriormente l'approccio rispetto al padding deterministico PKCS#1 v1.5; PSS è quindi da intendersi come raffinamento di livello produttivo del medesimo paradigma, non come primitiva alternativa.
3. **Funzioni di Hashing (SHA-256):** Utilizzate dall'AS per generare l'impronta digitale del token JSON prima di apporvi la firma, dall'EA per il calcolo degli identificativi delle ricevute e per l'indicizzazione del registro anti-replay, e (cfr. Sezione 2.8) per il concatenamento a hash-pointer della Bachecca Pubblica.

Confronto con le alternative. Per la cifratura, si è scartato l'*RSA grezzo* (deterministico, quindi non sicuro contro attacchi a testo in chiaro scelto e tale da rivelare informazione parziale sul messaggio) e il padding *PKCS#1 v1.5*, il cui padding randomico troppo corto offre garanzie solide solo in contesti limitati ed è storicamente esposto ad attacchi su oracolo; OAEP, introducendo seme casuale e *mask generation function*, rende i testi cifrati non malleabili. Per la firma, l'alternativa della *firma RSA grezza* è stata scartata perché vulnerabile a falsificazione senza messaggio, mentre *hash-and-sign* (RSA-FDH) raggiunge la stessa funzionalità al costo asintotico di una singola operazione a chiave privata, vincolando la firma a un'impronta resistente alle collisioni.

2.6 Strategia di Distribuzione delle Chiavi

Per ovviare alle problematiche di scalabilità e al rischio di intercettazione o sostituzione delle chiavi pubbliche da parte di avversari attivi sul canale, si adotta la seguente politica di distribuzione *Out-of-Band* basata su PKI federata:

- **Chiave Pubblica dell'AS (PK_{AS}):** Viene distribuita e codificata rigidamente all'interno dei server dell'Autorità Elettorale in fase di configurazione iniziale del sistema, riducendo a zero lo scambio dinamico di chiavi di validazione tra i due domini.
- **Chiave Pubblica dell'EA (PK_{EA}):** È veicolata da un **certificato X.509 intestato al dominio dell'EA**, emesso da un'Autorità di Certificazione pubblicamente fidata, e il client la scarica **direttamente dall'EA** su canale TLS, mai dal portale dell'AS. La chiave è inoltre ancorata a **due canali di verifica indipendenti**: (i) il **fingerprint SHA-256 di PK_{EA}** è la *prima entrata* della PBB, pubblicata e firmata al momento dell'indizione dell'elezione, prima dell'apertura delle urne; (ii) lo stesso fingerprint è riportato sul **decreto elettorale ufficiale** (canale istituzionale out-of-band, analogo all'identity proofing di SPID/CIE). Il client di voto, prima di cifrare, **verifica la catena di certificazione e confronta il fingerprint** della chiave ricevuta con quello pubblicato sulla PBB; in caso di mismatch la procedura di voto viene interrotta con segnalazione.

Perché PK_{EA} non deve transitare dall'AS. Questa scelta neutralizza un vettore d'attacco che la semplice fiducia nel TLS non copre: se la chiave dell'urna fosse iniettata nel client dal portale di Ateneo (design della versione precedente), un AS compromesso (*Evil Unisa*) potrebbe consegnare una chiave fraudolenta PK_X di cui detiene la privata. Poiché la PBB è *pubblica per requisito di verificabilità*, l'avversario potrebbe poi scaricare i ciphertext depositati e **decifrare ogni scheda**; conoscendo inoltre i **token_id** (che è in grado di derivare invocando l'HSM, sia pure lasciando traccia nei log), ricostruirebbe da solo l'intera mappatura identità→preferenza.

Allo scrutinio la frode emergerebbe (l'EA troverebbe schede indecifrabili, classificate come nulle in massa) ma la **segretezza, a differenza dell'integrità, non è ripristinabile a posteriori**: la rilevazione non è una difesa sufficiente. Con la distribuzione adottata, la sostituzione della chiave richiede di compromettere *simultaneamente* la catena di certificazione X.509 del dominio EA, la PBB (il cui contenuto è a sua volta verificato dagli auditor) e il decreto istituzionale: tre perimetri indipendenti, nessuno dei quali sotto il controllo dell'AS.

Confronto con le alternative. La distribuzione adottata è una forma di *public-key certificate* / PKI con scambio iniziale out-of-band, che protegge dalla sostituzione attiva delle chiavi sul canale. Le alternative studiate sono state scartate per ragioni precise: il *public announcement* è banalmente falsificabile da chiunque; la *public directory* e la *public-key authority* introducono un punto di consultazione online che è insieme collo di bottiglia e bersaglio; lo scambio *Diffie-Hellman* puro, in assenza di autenticazione delle chiavi, resta esposto ad attacchi *Man-in-the-Middle* (la nostra minaccia *Malcolm*); un *KDC* simmetrico, infine, costituirebbe un *single point of failure* la cui compromissione romperebbe l'intero sistema; la **consegna di PK_{EA} tramite il portale dell'AS** (alternativa scartata in questa revisione) concentrerebbe sul dominio d'identità un potere di compromissione della segretezza incompatibile con il Threat Model, come argomentato sopra. L'ancoraggio a certificati X.509 emessi da una CA fidata, il fingerprint impegnato sulla PBB e l'hardcoding di PK_{AS} presso l'EA neutralizzano questi vettori riconducendo la fiducia alla catena di certificazione *più* un canale di verifica pubblico e indipendente.

2.7 Flusso di Comunicazione tra le Parti

Il ciclo operativo completo della votazione segue una sequenza lineare di passi inter-nodo:

1. **Composizione Voto:** Il client scarica la chiave PK_{EA} dal dominio dell'EA, ne **verifica il certificato X.509 e il fingerprint** contro quello impegnato sulla PBB (Sezione 2.6), permette all'elettore di selezionare la lista, impacchetta la scheda in chiaro M , calcola il ciphertext $C = E_{PK_{EA}}(M)$ e invia il blocco B_{packet} all'EA.
2. **Verifica di Origine, Integrità e Ammissibilità:** L'EA riceve il pacchetto ed esegue la **verifica della firma** del token impiegando PK_{AS} . Se la firma è valida, l'EA verifica inoltre che **election_id** corrisponda all'elezione in corso e che **eligible_scope** sia compatibile con l'organo elettivo target; pacchetti con token di elezione errata o categoria non ammessa vengono scartati. Superati questi controlli, l'EA acquisisce la certezza matematica che il token è autentico, emesso dall'Università e valido per l'elezione corrente.
3. **Protezione Anti-Replay:** L'EA estrae il campo **token_id** e interroga il proprio **Registro dei Token Spesi**. Se l'identificativo è già presente, il pacchetto viene scartato.
4. **Archiviazione e Ricevuta:** In caso di esito positivo dei controlli, l'EA brucia definitivamente il **token_id** inserendolo nel registro e accoda il ciphertext C alla **Public Bulletin Board (PBB)**, strutturata come *registro tamper-evident*: ogni nuova entrata include un **hash-pointer** all'entrata precedente, così che qualsiasi alterazione, soppressione o riordino a posteriori delle schede già acquisite (minaccia *Gianfranco il traditore*) invaliderebbe l'intera catena successiva e sarebbe rilevabile pubblicamente. L'EA calcola quindi l'impronta della foglia depositata, la firma con la propria chiave privata (SK_{EA}) e la restituisce al browser dell'elettore come **Ricevuta di Voto**: essa identifica univocamente la posizione della scheda nel registro e ne consentirà la prova di inclusione (cfr. Sezione 2.9.1).

Sul furto del token. È necessario dichiarare apertamente che T è una credenziale *al portatore* (*bearer token*): l'EA ne verifica la firma dell'AS e l'unicità, ma (per preservare lo pseudoanonimato) ignora deliberatamente *chi* lo presenta. Ne consegue che un avversario che riesca a sottrarre un token **prima** che venga speso può effettivamente usarlo per votare.

Questo rischio è mitigato su più fronti senza reintrodurre legami con l'identità: (i) il token transita esclusivamente su canale TLS, il che neutralizza l'intercettazione di rete (minaccia *Malcolm*); (ii) il token è **monouso** grazie al Registro dei Token Spesi, dunque è utilizzabile *al più una volta*: se il legittimo elettore ha già votato, il tentativo del ladro viene rifiutato, e viceversa, se il ladro vota per primo, il successivo tentativo del legittimo elettore viene respinto, rendendo la frode **immediatamente rilevabile** tramite la verifica individuale (assenza della ricevuta attesa). Il vettore di furto residuo è quindi sostanzialmente la compromissione dell'*endpoint* (minaccia *Alessandria Project*, esterna al perimetro crittografico) o la coercizione (rischio residuo già accettato nel WP1). Una difesa più stringente (ad esempio legando il token a una chiave effimera detenuta dal client (*holder-of-key token*) o a un valore di *channel binding* TLS) è possibile ma reintrodurrebbe un'identità crittografica lato client, in tensione con lo pseudoanonimato di rete; viene pertanto indicata come estensione futura.

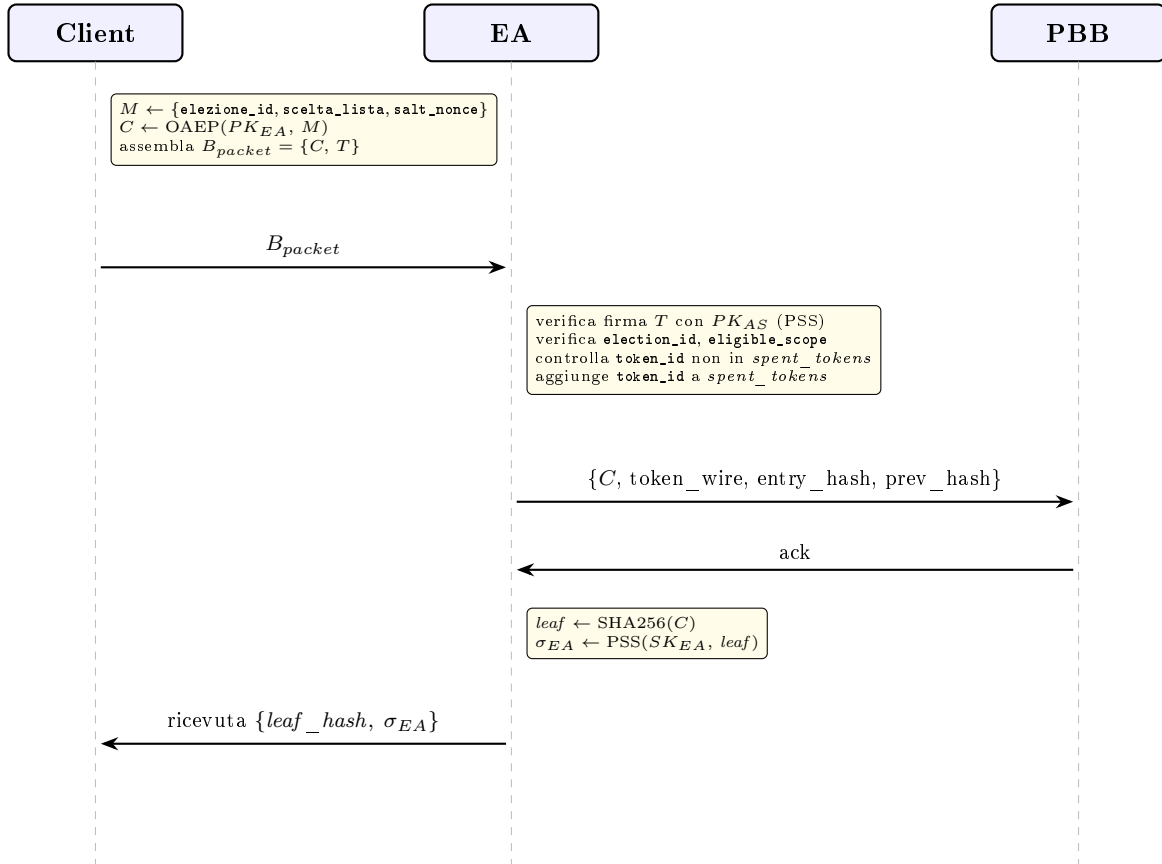


Figura 3: Diagramma di sequenza. Flusso di sottomissione del voto (§2.7).

2.8 Procedura di Scrutinio e Pubblicazione del Risultato

La procedura di spoglio unisce la necessità di trasparenza con la salvaguardia della segretezza a urne chiuse:

1. **Chiusura dei Seggi e Sigillo dell'Urna:** Al raggiungimento dell'orario limite stabilito nel timestamp del token, l'AS invalida l'emissione di nuovi token e l'EA interrompe l'accettazione di qualsiasi pacchetto di sottomissione. Il contenuto della PBB viene congelato; l'EA calcola la **radice di un albero di Merkle** costruito su tutte le schede cifrate acquisite, la firma con SK_{EA} e la pubblica. Questa radice è un *impegno crittografico* (*commitment*) sull'insieme esatto delle schede in ingresso: ne fissa pubblicamente cardinalità e contenuto *prima* dello scrutinio, impedendo all'EA di aggiungere o sopprimere schede in un secondo momento.
2. **Shuffling (Rimescolamento Anonimo):** Per ostacolare attacchi di linkability basati sull'analisi temporale dei log di rete, l'EA sottopone l'intero blocco delle schede cifrate a un algoritmo di **rimescolamento randomico** (es. Fisher-Yates), che altera l'ordine dei ciphertext rispetto alla sequenza di ricezione. Va precisato che si tratta di una **misura ingegneristica di anonimizzazione, non di una primitiva crittograficamente verificabile**: di per sé un rimescolamento in chiaro non dimostra a un terzo che la permutazione preservi l'insieme delle schede (su questo limite si torna più avanti).
3. **Decifratura a Soglia:** La chiave privata dell'Autorità Elettorale (SK_{EA}) non è detenuta intera da alcun soggetto, ma è suddivisa tra i membri della commissione elettorale tramite uno schema di **condivisione del segreto di Shamir** (t, n): servono almeno t degli n membri per ricostruirla, mentre qualsiasi sottoinsieme di dimensione inferiore non apprende **alcuna** informazione su SK_{EA} . Solo al raggiungimento del quorum t , e all'interno di un ambiente di computazione isolato, la chiave viene ricomposta e l'EA esegue la decifratura in batch delle schede rimescolate.
4. **Validazione e Pubblicazione:** Per ogni scheda decifrata l'EA verifica la **buona formazione** del JSON (coerenza dell'`elezione_id` e appartenenza di `scelta_lista` all'insieme delle liste ammesse); le schede non conformi (cfr. Sezione 2.4) sono marcate come **nulle** e contate separatamente. L'EA pubblica infine sulla PBB l'elenco dei voti in chiaro rimescolati e l'esito della validazione, consentendo a qualunque osservatore il conteggio aggregato trasparente dei seggi. Pubblica, inoltre, anche l'elenco dei voti non conteggiati (schede nulle), per garantire la consultabilità da parte degli elettori, in contrasto con i potenziali attacchi interni (*Alessandria Project*). La pubblicazione avviene esclusivamente dopo il rimescolamento e senza conservare alcuna informazione sull'ordine di arrivo delle schede; in contesti con un numero molto ridotto di votanti si dovrebbe valutare l'adozione di soglie minime di pubblicazione o l'aggregazione dei risultati, per ridurre il rischio di inferenze statistiche sulla preferenza individuale.

Limite di verificabilità e gestione della fiducia. L'impegno di Merkle del punto 1 garantisce che l'*insieme in ingresso* sia fissato pubblicamente e che la sua **cardinalità si conservi** (gli auditor verificano che il numero di schede impegnate eguagli quello dei voti pubblicati, validi più nulli). Tuttavia, un rimescolamento in chiaro seguito da decifratura in batch **non fornisce una prova crittografica** che la permutazione e la decifratura siano corrette scheda per scheda: un'EA disonesta potrebbe, in linea di principio, sostituire il contenuto di una scheda mantenendone invariata la cardinalità. Rimuovere completamente questa assunzione di fiducia richiederebbe strumenti che esulano dal programma del corso (in particolare *mix-net verificabili* con prove a conoscenza zero della permutazione, oppure una decifratura a soglia corredata da prove di corretta decifratura) che indichiamo come naturale **estensione futura**.

Allo stato, dunque, la verificabilità universale è garantita sulla *conservazione e l'integrità dell'insieme* e poggia, per la *correttezza del singolo passaggio di rimescolamento-decifratura*, su un'assunzione di fiducia nell'EA esplicitata nella Sezione 2.11.2.

2.9 Meccanismi di Verifica Individuale e Universale

2.9.1 Verificabilità Individuale

La Bacheca Pubblica è accessibile liberamente e senza autenticazione da parte di chiunque: non è necessario alcun token per consultarla. L'eliminazione del token di osservazione semplifica il protocollo e azzera il rischio di *linkability* nella fase di consultazione: non essendo presentato alcun identificativo all'EA durante la lettura della PBB, non esiste canale attraverso cui collegare la sessione di consultazione al voto precedentemente espresso.

L'elettore che desideri verificare l'inclusione della propria scheda presenta la **ricevuta firmata** rilasciata dall'EA al momento della sottomissione. La verifica procede in due passi:

1. Verifica della firma PSS di σ_{EA} su *leaf_hash* tramite PK_{EA} : certifica che la foglia è stata effettivamente registrata dall'EA.
2. Verifica della **prova di inclusione Merkle**: il percorso dalla foglia alla radice viene ricalcolato e confrontato con la radice firmata e pubblicata al momento del sigillo dell'urna.

Il riscontro positivo fornisce la certezza matematica che il **ciphertext associato alla ricevuta** è presente e inalterato nell'urna elettronica. Va precisato che questa prova garantisce l'inclusione della scheda cifrata, non la correttezza della codifica della preferenza né la correttezza della decifratura finale: questi ultimi aspetti ricadono rispettivamente nell'assunzione di integrità del dispositivo client (cfr. minaccia *Alessandria Project*) e nell'assunzione di correttezza dello scrutinio dell'EA. Vengono parzialmente mitigati questi rischi attraverso l'introduzione dei *leaf_hash* relativi alle schede nulle.

2.9.2 Verificabilità Universale

Chiunque ha il diritto di scaricare l'intero dump della bacheca PBB. Gli auditor indipendenti eseguono script automatizzati per:

1. Verificare l'**integrità del registro**: ricalcolare la catena di hash-pointer e la radice di Merkle dell'urna, controllando che coincidano con la radice firmata dall'EA al momento del sigillo, così da escludere alterazioni, soppressioni o riordini delle schede.
2. Verificare l'**ammissibilità**: controllare che ogni scheda cifrata accodata sia corredata dal relativo token anonimo (*token_id* e firma) recante una firma autentica dell'AS, validabile con PK_{AS} , senza che ciò riveli alcuna identità.
3. Verificare la **conservazione e il conteggio**: accertare che il numero di voti pubblicati (validi più nulli) eguagli la cardinalità impegnata nella radice di Merkle e ricalcolare in autonomia il **conteggio** aggregato dei voti partendo dai dati in chiaro pubblicati post-scrutinio, validando la corrispondenza con il report finale proclamato dalla commissione elettorale.

2.10 Meccanismo di Sostituzione o Annullamento del Voto

In perfetta concordanza con le valutazioni dei requisiti esposte nel WP1, l'architettura del protocollo **non prevede in alcun modo la revocabilità o la sostituzione del voto** prima della chiusura formale delle urne.

Tale feature avrebbe esposto il sistema a vulnerabilità critiche, in particolare la Coercizione e il Voto di Scambio. La scelta di non consentire la sostituzione del voto **riduce alcuni scenari di manipolazione successiva** e limita l'utilità del furto di credenziali dopo l'espressione del voto: un attaccante non potrà usare credenziali rubate per sovrascrivere un voto già espresso legittimamente. Tuttavia, questa scelta **non elimina** la coercizione esercitata *durante* l'atto di voto, che resta un rischio residuo tipico del voto elettronico remoto in ambiente non controllato fisicamente. La non-revocabilità va quindi intesa come una mitigazione parziale di scenari specifici di compravendita, non come una difesa completa contro la coercizione.

2.11 Analisi di Efficienza e Trasparenza

2.11.1 Efficienza Computazionale e di Rete

La soluzione proposta risponde in modo ottimale al vincolo di efficienza imposto:

- I controlli di unicità espletati dall'EA sul Registro dei Token Spesi avvengono a tempo costante $\mathcal{O}(1)$, garantendo latenze minime ed escludendo colli di bottiglia computazionali durante i picchi di affluenza.
- Il client esegue una sola operazione di **cifratura con chiave pubblica** (RSA-OAEP), computazionalmente meno onerosa rispetto alla generazione di firme, alla gestione di chiavi private locali e alla decifratura massiva. Il carico più significativo — rappresentato dalla decifratura delle schede e dallo scrutinio — è confinato alla fase post-elettorale e gestito interamente dall'Autorità Elettorale in ambiente isolato, preservando così l'usabilità del sistema anche su dispositivi mobili o hardware datati.

2.11.2 Trasparenza e Assunzioni di Fiducia

In conformità con i vincoli di trasparenza, si dichiarano esplicitamente le componenti che richiedono assunzioni di fiducia e i relativi rischi mitigati:

1. **Fiducia nell'Integrità dell'AS:** Si assume che il Server di Autenticazione sia onesto e non emetta token validi a identificativi inesistenti (generazione di elettori fantasma). Tale assunzione è motivata dal fatto che l'AS è gestito direttamente dall'infrastruttura centrale dell'Università degli Studi di Salerno, la quale risponde a audit interni rigorosi.
2. **Fiducia sul Canale di Rete (TLS):** Si assume la sicurezza dei canali di comunicazione cifrati tra client, AS ed EA. Il rischio di attacchi *Man-in-the-Middle* è mitigato dalle policy HSTS e dal *Certificate Pinning* nel client nativo di voto.
3. **Fiducia nella Correttezza dello Scrutinio dell'EA:** Si assume che l'EA esegua onestamente il rimescolamento e la decifratura delle schede. Come discusso nella Sezione 2.8, l'impegno di Merkle sull'urna e la decifratura a soglia (t, n) riducono fortemente questo rischio (garantendo rispettivamente la conservazione dell'insieme delle schede e l'impossibilità per un singolo membro della commissione di violare la segretezza) ma non forniscono una prova crittografica della correttezza della permutazione e della decifratura scheda per scheda. Il rischio residuo di una sostituzione di contenuto a cardinalità invariata da parte di un'EA pienamente compromessa è affidato agli audit della commissione e alle tutele legali, e la sua eliminazione è demandata all'adozione di una *mix-net verificabile* come estensione futura.

4. **Fiducia nella Custodia di K_{HMAC} :** La proprietà di non collegabilità identità $\leftrightarrow$ scheda da parte del solo AS poggia sull'assunzione che K_{HMAC} resti confinata nell'HSM e che il quorum t della commissione di garanzia non venga raggiunto fraudolentemente (Sezione 2.2). Il rischio residuo (compromissione fisica del modulo hardware o collusione della commissione) è di natura organizzativa, rilevabile tramite il log firmato delle invocazioni, ed è affidato agli audit.
5. **Fiducia nella Catena di Certificazione per PK_{EA} :** L'autenticità della chiave dell'urna poggia sulla CA pubblica che certifica il dominio EA e sulla coerenza del fingerprint pubblicato su PBB e decreto (Sezione 2.6); la compromissione simultanea di entrambi i canali è considerata fuori scopo per gli avversari del Threat Model.

3 WP3 - Analisi della Sicurezza

Questo work package analizza la sicurezza della soluzione progettata nel WP2 rispetto al modello definito nel WP1. Coerentemente con il mandato della traccia, l'analisi persegue due obiettivi distinti: **(i)** verificare, proprietà per proprietà, in che misura il protocollo resista agli avversari del Threat Model; **(ii)** accertare che non esistano modifiche ovvie capaci di migliorare alcune proprietà *senza alcuna perdita* nelle altre.

3.1 Metodologia dell'Analisi

L'analisi è guidata dalle proprietà di sicurezza (Sezione 3.2), ma ciascuna proprietà viene chiusa esplicitamente contro gli avversari nominati nel WP1; la Sezione 3.3 ripercorre poi l'intero Threat Model dal punto di vista degli attaccanti, a garanzia di copertura completa. Ogni proprietà è discussa secondo lo stesso schema fisso: **garanzia** offerta, **meccanismo** realizzativo del WP2, **avversari neutralizzati**, **assunzioni** di fiducia e **rischio residuo**.

Dichiariamo inoltre la gerarchia di priorità adottata, come richiesto dalla traccia: vengono privilegiate per prime **integrità**, **autenticità** e **unicità** (il nucleo della correttezza dell'esito elettorale), quindi **segretezza** e **pseudoanonimato**, infine la **verificabilità**; **disponibilità** e **coercizione** sono trattate come vincoli a sé stanti. Alcune proprietà risultano garantite in modo incondizionato, altre solo sotto le assunzioni dichiarate: di seguito le distinguiamo con precisione.

3.2 Analisi delle Proprietà di Sicurezza

3.2.1 Autenticità

Garanzia. Solo gli aventi diritto, autenticati tramite credenziali verificabili, ottengono un token di voto valido. **Meccanismo.** Identità federata SPID/CIE (con l'AS in veste di *Relying Party*), verifica del diritto sul database degli iscritti ed emissione del token T firmato in *hash-and-sign* e verificabile dall'EA tramite PK_{AS} . **Avversari neutralizzati.** *Votazioni.UMISA.IT* (phishing): l'*identity proofing* out-of-band di SPID/CIE e il canale TLS con HSTS e *Certificate Pinning* impediscono il dirottamento delle credenziali, e un token contraffatto fallirebbe comunque la verifica di firma presso l'EA. **Assunzioni.** Integrità dell'*Identity Provider* nazionale e onestà dell'AS (Sezione 2.11.2). **Rischio residuo.** *Evil Unisa* (AS pienamente compromesso) può emettere token per soggetti del proprio bacino anagrafico; questo residuo è discusso nella Sezione 3.4.

3.2.2 Unicità

Garanzia. Ciascun avente diritto esprime al più un voto valido. **Meccanismo.** La derivazione deterministica $\text{token_id} = \text{HMAC}(SK_{AS}, \text{identity} \parallel \text{election_id})$ garantisce che lo stesso elettore presenti sempre lo stesso token_id ; il **Registro dei Token Spesi** dell'EA è il presidio principale: ogni token_id può essere speso al più una volta ($\mathcal{O}(1)$). Il flag lato AS offre difesa in profondità: impedisce l'emissione di token con token_id diversi per lo stesso elettore nella stessa elezione, anche se l'AS è la sola a conoscere SK_{AS} . **Avversari neutralizzati.** *Alberto l'Einsteinio* (doppio voto): il secondo tentativo di sottomissione viene rifiutato dall'EA perché il token_id è già in *spent_tokens*; il replay di *Malcolm* è neutralizzato dalla stessa proprietà monouso. **Assunzioni.** Integrità del Registro dei Token Spesi dell'EA e onestà dell'AS nel non rivelare SK_{AS} . **Rischio residuo.** Un AS compromesso (*Evil Unisa*) potrebbe generare token per soggetti non aventi diritto; coincide con il residuo dell'Autenticità.

3.2.3 Segretezza

Garanzia. La preferenza del singolo elettore non è rivelabile, neppure ai gestori del sistema, durante l'elezione. **Meccanismo.** Cifratura RSA-OAEP verso PK_{EA} (semanticamente sicura e non malleabile); l'AS è cieco rispetto alla scheda; la decifratura avviene solo in forma aggregata, a urne chiuse, con SK_{EA} ricostruita a soglia (t, n) in ambiente isolato. **Avversari neutralizzati.** *Gianfranco* (accesso al database) osserva unicamente testi cifrati; un singolo amministratore dell'EA non può anticipare la decifratura, grazie alla condivisione del segreto di Shamir; *Evil Unisa* nella variante **sostituzione della chiave dell'urna** è neutralizzato dalla distribuzione indipendente di PK_{EA} (certificato X.509 del dominio EA, fingerprint impegnato sulla PBB e sul decreto, verifica obbligatoria lato client, Sezione 2.6): l'AS non si trova più sul percorso di distribuzione e non può consegnare una PK_X fraudolenta; *Asse Learning-Election* è trattato sotto Pseudoanonimato. **Assunzioni.** Difficoltà del problema di fattorizzazione (sicurezza di RSA), onestà di un numero di membri della commissione inferiore alla soglia t , integrità della catena di certificazione del dominio EA. **Rischio residuo.** Collusione totale AS+EA; coercizione all'istante del voto (rischio residuo esplicitamente accettato nel WP1).

3.2.4 Pseudoanonimato

Garanzia. Nessuna entità singola è in grado di legare l'identità civile alla preferenza espressa, sotto l'assunzione di custodia di K_{HMAC} . **Meccanismo.** Disaccoppiamento asimmetrico (l'AS conosce il "chi", l'EA il "cosa"), assenza di canali server-to-server, accesso alla PBB completamente pubblico e senza token (nessun identificativo presentato per la consultazione), rimescolamento delle schede allo scrutinio e custodia di K_{HMAC} in HSM non esportabile con backup a soglia di Shamir e log firmato (Sezione 2.2). **Avversari neutralizzati.** *Asse Learning-Election*: la de-anonimizzazione richiede di compromettere due perimetri indipendenti. Quanto alla derivazione HMAC, l'analisi è dichiarata con la **stessa severità** in tutto il documento: il detentore di K_{HMAC} collegherebbe *da solo* identità e scheda cifrata, senza alcuna correlazione temporale: un attacco **qualitativamente più forte** di quello a token casuale, che richiedeva la collusione di entrambi i domini. Il vettore è neutralizzato *a livello software*: la compromissione applicativa dell'AS non restituisce la chiave (l'HSM espone la sola primitiva HMAC) e un ricalcolo massivo dei `token_id` produrrebbe un volume di invocazioni anomalo sul log firmato, rilevabile in audit. **Assunzioni.** Separazione amministrativa e infrastrutturale dei due domini (Sezioni 2.4 e 2.11.2); confinamento di K_{HMAC} nel perimetro hardware; assenza di quorum fraudolento nella commissione di garanzia. **Rischio residuo.** Collusione coordinata tra le due organizzazioni, oppure violazione fisica dell'HSM / collusione della commissione di garanzia: entrambi gli scenari sono di natura organizzativa e affidati alle tutele legali post-elettorali e agli audit sul log delle invocazioni.

3.2.5 Integrità

Garanzia. I voti non sono alterabili, sopprimibili o introducibili fraudolentemente dopo l'espressione. **Meccanismo.** Token in *hash-and-sign* (ogni manomissione invalida la firma); PBB *tamper-evident* a hash-pointer con *commitment* di Merkle al momento del sigillo (qualsiasi alterazione o riordino successivo rompe la catena); ricevuta firmata. Quanto al transito, va precisato con rigore il ruolo di ciascun presidio: la **non malleabilità di OAEP** impedisce di trasformare un ciphertext in uno di un plaintext correlato, ma, non essendo OAEP uno schema di autenticazione, non impedisce a chi disponga di un token valido di *sostituire integralmente* il ciphertext con uno proprio. La protezione contro alterazione e sostituzione in transito è quindi affidata congiuntamente al **canale TLS** (riservatezza e integrità del trasporto), alla **firma del token** (un B_{packet} con token manomesso è scartato) e alla proprietà **monouso** del token (un avversario privo di un token fresco non può iniettare schede).

Avversari neutralizzati. *Gianfranco* (modifica o cancellazione dei voti archiviati) è rilevato dalla catena di hash-pointer e dal commitment; *Malcolm* (iniezione o alterazione in transito) è neutralizzato da TLS, dalla firma del token e dal registro anti-replay. **Assunzioni.** Resistenza alle collisioni di SHA-256 e sicurezza dello schema di firma. **Rischio residuo.** *Alessandria Project* (alterazione del voto sull'*endpoint*, prima della cifratura) **non** è neutralizzata: si tratta di una compromissione del client, esterna al perimetro crittografico, ripresa nella Sezione 3.4.

3.2.6 Verificabilità Individuale

Garanzia. Ogni elettore verifica autonomamente che il proprio voto sia incluso nel conteggio. **Meccanismo.** Ricevuta firmata dall'EA (σ_{EA} su *leaf_hash*) e prova di inclusione Merkle, confrontata con la radice firmata. La PBB è pubblica: nessun token è richiesto per la consultazione, eliminando il rischio di *linkability* nella fase di verifica. **Avversari neutralizzati.** Un *Gianfranco* che tenti la soppressione silenziosa di una scheda viene rilevato (la prova di inclusione risulterebbe assente); il furto di un token spende il voto, e l'assenza della ricevuta attesa segnala la frode al legittimo elettore. **Assunzioni.** Pubblicazione onesta della radice di Merkle (legata al rischio residuo dell'EA). **Rischio residuo.** Il *cast-as-intended* (il testo cifrato codifica effettivamente la scelta voluta) non è verificabile dal solo elettore senza meccanismi aggiuntivi: ricade sotto *Alessandria Project*. L'elettore può solo verificare che il proprio voto non sia stato reso nullo mediante la consultazione dei *leaf_hash* relativi ai voti nulli.

3.2.7 Verificabilità Universale

Garanzia. Chiunque può verificare l'integrità del registro pubblico, la conservazione della cardinalità delle schede, l'ammissibilità dei token anonimi e la correttezza del conteggio sui voti pubblicati. La correttezza crittografica del passaggio di rimescolamento e decifratura scheda per scheda resta invece affidata all'EA, salvo adozione futura di mix-net verificabili o prove a conoscenza zero. **Meccanismo.** Scaricamento del dump della PBB; verifica della catena di hash-pointer e della radice di Merkle (integrità), delle firme dell'AS sui token anonimi (ammissibilità), della conservazione della cardinalità (schede impegnate = voti validi + nulli) e ricalcolo autonomo del conteggio. **Avversari neutralizzati.** *Gianfranco* (aggiunta o soppressione di schede) è rilevato dalla verifica di integrità del registro e dalla conservazione della cardinalità. **Assunzioni.** Onestà dell'EA nell'esecuzione dello scrutinio (Sezione 2.11.2). **Rischio residuo.** È il limite principale e dichiarato: la correttezza *scheda-per-scheda* del rimescolamento e della decifratura non è dimostrata crittograficamente, lasciando spazio a una sostituzione di contenuto a cardinalità invariata da parte di un'EA compromessa. La sua eliminazione richiede una *mix-net verificabile* o una decifratura a soglia con prove a conoscenza zero (Sezione 3.4).

3.2.8 Disponibilità

Garanzia. Gli aventi diritto possono esprimere il voto senza interruzioni. **Meccanismo.** Controlli di unicità a tempo costante $\mathcal{O}(1)$, operazioni crittografiche leggere lato client, decifratura pesante confinata asincronamente al post-scrutinio e separazione AS/EA. **Avversari neutralizzati.** Va dichiarato con onestà che, contro *Gruppo Blackwatch* (DDoS), la crittografia non risolve il problema: le contromisure sono infrastrutturali e di processo (*rate-limiting*, replica geografica dei servizi, CDN, code di ingresso virtuali in caso di picchi, filtraggio del traffico anomalo, backup della PBB, monitoraggio durante la finestra elettorale e possibilità di estendere la finestra in caso di indisponibilità documentata). La separazione AS/EA limita la propagazione: un DoS sull'AS non impedisce la verifica sulla PBB, e viceversa. **Assunzioni.** Dimensionamento e ridondanza adeguati dell'infrastruttura. **Rischio residuo.** Un attacco volumetrico sufficientemente esteso resta una minaccia; è un compromesso intrinseco al voto elettronico remoto, non un difetto delle primitive crittografiche.

3.3 Chiusura del Threat Model

La Tabella 1 ripercorre l'intero Threat Model del WP1 dal punto di vista degli avversari, riepilogando per ciascuno la difesa apprestata e l'esito. Le righe contrassegnate come *rischio residuo* introducono i compromessi discussi nella sezione successiva.

Avversario	Proprietà bersaglio	Difesa apprestata ed esito
Alberto l'Einsteinio	Unicità	Flag lato AS e Registro dei Token Spesi ($O(1)$); token monouso. Neutralizzato .
Gianfranco il traditore	Integrità (voti archiviati)	Catena di hash-pointer, commitment di Merkle e congelamento della PBB. Neutralizzato (manomissione rilevabile).
Evil Unisa	Autenticità / Unicità / Segretezza	Ancoraggio all'IdP nazionale limita le identità sintetiche; la sostituzione di PK_{EA} è neutralizzata dalla distribuzione indipendente con fingerprint su PBB e decreto (§2.6); resta possibile l'abuso sul bacino interno. Rischio residuo (audit interni).
Asse Learning-Election	Pseudoanonimato / Segretezza	Separazione dei domini, K_{HMAC} confinata in HSM con backup Shamir e log firmato, rimescolamento. Mitigato (richiede collusione totale o violazione del perimetro hardware).
Malcolm in the Middle	Intercettazione / Unicità	TLS con HSTS e pinning, firma del token, monouso. Neutralizzato .
Votazioni.UMISA.IT	Phishing / Autenticità	Identity proofing out-of-band SPID/CIE su canale TLS. Mitigato (residuo sul fattore umano).
Gruppo Blackwatch	Disponibilità	Mitigazioni infrastrutturali (rate-limiting, replica, CDN); separazione AS/EA. Mitigato (DoS non risolvibile crittograficamente).
Alessandria Project	Integrità (pre-cifratura)	Esterno al perimetro crittografico; non neutralizzabile lato protocollo. Rischio residuo (<i>cast-as-intended</i>).

Tabella 1: Chiusura del Threat Model: mappatura avversario $\rightarrow$ proprietà $\rightarrow$ difesa ed esito.

3.4 Modifiche a Costo Nullo e Compromessi Strutturali

Il secondo mandato del WP3 richiede di verificare che non esistano modifiche ovvie capaci di migliorare alcune proprietà senza perdite nelle altre. Affrontiamo la questione in due tempi: prima le migliorie a costo nullo che l'analisi ha fatto emergere e che sono state integrate, poi i compromessi che *non* sono gratuiti e che motivano i limiti residui.

3.4.1 Migliorie individuate e integrate

L'analisi ha messo in luce alcune modifiche con beneficio netto e **senza perdita**, già recepite nella versione finale del WP2:

- **AS che non persiste il `token_id`**: rafforza lo pseudoanonimato (rimuove l'unica mappatura che legherebbe i due domini) a costo nullo;
- **PBB a hash-pointer con commitment di Merkle**: rende l'integrità del registro crittograficamente imposta e fornisce alla verificabilità individuale una prova di inclusione, con overhead trascurabile;
- **Decifratura a soglia di Shamir su SK_{EA}** : elimina il *single point of compromise* sulla segretezza, impedendo a un singolo amministratore di aprire l'urna, senza alcuna perdita per le altre proprietà;
- **Validazione di buona formazione delle schede**: gestisce in modo trasparente i testi cifrati malevoli che decifrano in valori non validi, classificandoli come nulli, senza impatto sulle proprietà preesistenti;

- **Distribuzione indipendente di PK_{EA}** (integrata nella revisione v3): rimuovere l'AS dal percorso di distribuzione della chiave dell'urna, ancorandola a certificato X.509 del dominio EA e a un fingerprint impegnato su PBB e decreto, elimina il vettore di sostituzione della chiave da parte di *Evil Unisa senza alcuna perdita* nelle altre proprietà: il costo è una verifica di fingerprint lato client, computazionalmente trascurabile;
- **Custodia in HSM con backup a soglia di K_{HMAC}** (integrata nella revisione v3): trasforma la linkabilità identità $\leftrightarrow$ token_id da rischio software a rischio fisico-organizzativo auditabile. Non è strettamente a costo nullo (richiede un modulo hardware e una cerimonia delle chiavi) ma non sacrifica alcuna proprietà di sicurezza né l'efficienza per-votante, ed è quindi un miglioramento netto sul piano del protocollo.

3.4.2 Compromessi non gratuiti

I gap residui evidenziati nella Sezione 3.2 **non** sono colmabili a costo nullo: ogni miglioria richiederebbe il sacrificio di un'altra proprietà o l'impiego di strumenti esterni al programma del corso.

- **Mix-net verificabile** o decifratura a soglia con prove a conoscenza zero: chiuderebbe completamente la verificabilità universale (rimuovendo la fiducia sullo scrutinio dell'EA), ma a costo di efficienza e di strumenti non trattati a lezione. *Estensione futura.*
- **Resistenza alla coercizione** (re-voting, credenziali fittizie in stile JCJ): migliorerebbe la resistenza alla coercizione, ma a prezzo di efficienza ed esperienza utente, ed è in conflitto con la scelta di irrevocabilità. *Rinviata.*
- **Revocabilità o sostituzione del voto**: offrirebbe correzione d'errore, ma riaprirebbe gli attacchi di coercizione e voto di scambio: compromesso già valutato e respinto nel WP1 (Sezione 2.4 e relativa discussione sulla non-revocabilità).
- **Holder-of-key token**: renderebbe il token resistente al furto, ma reintrodurrebbe un'identità crittografica lato client, erodendo lo pseudoanonimato di rete. *Estensione futura.*
- **Firma cieca**: occulterebbe il token_id anche all'EA, ma non è materiale del corso e il modello a token firmato dall'AS già fornisce la proprietà richiesta. *Menzione come estensione.*

3.5 Conclusione

L'analisi mostra che le proprietà definite nel WP1 sono **soddisfatte o mitigate** entro le assunzioni di fiducia dichiarate. In particolare, **autenticità, unicità e integrità del registro** risultano fortemente garantite dai meccanismi di firma hash-and-sign, anti-replay e append-only log tamper-evident. **Segretezza e pseudoanonimato** dipendono dalla separazione infrastrutturale tra AS ed EA, dal rispetto della politica di logging e, nella presente revisione, da due presidi aggiuntivi che chiudono i vettori più gravi della versione precedente: la **distribuzione indipendente di PK_{EA}** , che impedisce a un AS compromesso di sostituire la chiave dell'urna e decifrare le schede dalla PBB pubblica, e la **custodia in HSM con backup a soglia di K_{HMAC}** , che impedisce alla compromissione del solo software dell'AS di ricostruire la mappatura identità $\leftrightarrow$ scheda. La **verificabilità universale** è garantita rispetto all'integrità del registro, alla conservazione della cardinalità dell'urna e alla correttezza del conteggio sui voti pubblicati, ma non elimina completamente la fiducia nella correttezza del rimescolamento e della decifratura: questo residuo è esplicitato nella Sezione 2.11.2 e motivato dall'assenza di mix-net verificabili nel programma del corso. Le migliorie integrate nella Sezione 3.4 rispondono al secondo requisito della traccia producendo benefici dimostrabili senza perdite nelle proprietà restanti.

I gap residui non sono colmabili gratuitamente: oltre al limite sulla verificabilità universale, la soluzione non può impedire che un **client compromesso** (*Alessandria Project*) alteri la preferenza prima della cifratura. Tale rischio non deriva da una debolezza delle primitive crittografiche, ma dall'impossibilità di controllare il dispositivo dell'elettore in un sistema di voto remoto. I limiti residui sono dichiarati esplicitamente, con **la medesima valutazione di severità in ogni sezione del documento**, e risultano coerenti con i compromessi progettuali adottati.

4 WP4 - Implementazione e Prestazioni

Il presente work package realizza il protocollo descritto nel WP2 in un **ambiente simulato**: un'applicazione stand-alone eseguibile su un singolo computer, in cui le interazioni di rete tra le parti oneste (AS, EA, Client, PBB) sono modellate come chiamate dirette tra moduli. Coerentemente con la Nota 4.1, l'implementazione aderisce al WP2; l'unica deviazione formale dalla specifica iniziale (la pubblicazione del token anonimo nella PBB) è stata recepita nelle Sezioni 2.4 e seguenti. I sorgenti completi sono disponibili nel repository:

<https://github.com/SambucoS/Project-Work-APS>

4.1 Scelta Tecnologica

L'implementazione è scritta in **Python 3** e utilizza **esclusivamente** la libreria crittografica **cryptography** (PyCA). La scelta è motivata su tre piani: **conformità**, poiché PyCA è una libreria riconosciuta e ben mantenuta, in linea con il vincolo che impone di non impiegare primitive crittografiche custom; **sicurezza**, in quanto fornisce implementazioni standard e auditabili di RSA-OAEP, RSA-PSS e SHA-256, evitando i rischi di una reimplementazione manuale; **efficienza di sviluppo e misura**, grazie alla leggibilità del linguaggio e alla strumentazione immediata dei tempi tramite `time.perf_counter`. Le uniche componenti realizzate da zero sono quelle non crittografiche in senso stretto (catena di hash-pointer, albero di Merkle, condivisione del segreto di Shamir), costruite comunque sopra le primitive standard della libreria.

4.2 Architettura del Prototipo

Il sistema è organizzato in otto moduli che rispecchiano gli attori e le strutture dati del WP2 (Tabella 2). La separazione è funzionale: `crypto_utils.py` concentra l'uso di PyCA, mentre i moduli degli attori (AS, EA, Client) implementano la logica di protocollo richiamando tali primitivi.

Modulo	Responsabilità
<code>crypto_utils.py</code>	Generazione chiavi RSA-4096, OAEP, PSS, SHA-256 e (de)serializzazione PEM/DER, interamente via PyCA.
<code>pbb.py</code>	Public Bulletin Board: catena di hash-pointer, albero di Merkle e prove di inclusione.
<code>shamir.py</code>	Condivisione del segreto (t, n) su $GF(2^{127} - 1)$ tramite interpolazione di Lagrange, applicata ai byte di SK_{EA} .
<code>as_server.py</code>	Authentication Server: verifica eleggibilità, derivazione <code>token_id</code> via HMAC-SHA256, emissione token firmato, flag anti-duplicato per elezione.
<code>ea_server.py</code> <code>client.py</code>	Electoral Authority: intake dei voti, anti-replay $\mathcal{O}(1)$, pipeline di scrutinio. Client votante: cifratura della scheda, costruzione del B_{packet} , verifica della ricevuta e della prova di inclusione.
<code>simulation.py</code>	Dimostrazione end-to-end delle sei fasi del protocollo.
<code>benchmarks.py</code>	Misure di tempo e di dimensione dei messaggi.

Tabella 2: Mappa dei moduli dell'implementazione.

4.3 Esecuzione del Prototipo

Il prototipo può essere eseguito in ambiente locale con i comandi seguenti. L'esecuzione di `simulation.py` produce una dimostrazione completa del flusso di voto dalla generazione dei token alla pubblicazione sulla PBB, fino alla verifica individuale e al conteggio finale; `benchmarks.py` misura i costi computazionali e le dimensioni dei messaggi.

```

git clone https://github.com/SambucoS/Project-Work-APS
cd Project-Work-APS
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate  # Linux/macOS
pip install cryptography
python simulation.py        # flusso completo (default: 10 votanti)
python simulation.py --voters 50
python benchmarks.py

```

Listing 6: Comandi di installazione ed esecuzione

4.4 Decisioni Implementative Rilevanti

Le scelte seguenti, conformi al WP2, sono state imposte direttamente nel codice e meritano documentazione esplicita.

- **Condivisione del segreto (shamir.py).** Lo schema opera sul campo $\text{GF}(p)$ con $p = 2^{127} - 1$ (primo di Mersenne). La chiave SK_{EA} è serializzata in DER (PKCS8), interpretata come intero e suddivisa in *chunk* da 15 byte (120 bit, sotto la capacità utile per elemento di campo); ogni chunk è condiviso indipendentemente con un polinomio di grado $t - 1$, mentre lo stesso valore x identifica lo shareholder su tutti i chunk. La ricostruzione avviene per interpolazione di Lagrange in $f(0)$, con inverso modulare calcolato tramite il piccolo teorema di Fermat ($d^{p-2} \bmod p$) ed esattamente t share. Coefficienti e ascisse sono estratti dal CSPRNG `secrets`. *Nota di produzione:* lo splitting dei byte grezzi ricostruisce l'intera chiave in un solo punto; un sistema reale adotterebbe uno schema di *threshold RSA* (Shoup) in cui la chiave non viene mai riassemblata.
- **Serializzazione delle chiavi (crypto_utils.py).** Chiavi private in PKCS8 PEM non cifrate e pubbliche in SubjectPublicKeyInfo PEM. *Caveat di produzione:* in un contesto reale le chiavi private andrebbero protette con passphrase o custodite in un HSM; qui sono in chiaro al solo scopo di rendere la demo autocontenuta.
- **Formato del token (as_server.py).** Formato *wire*: `token_json || '.' || base64url(firma_PSS)`, con JSON serializzato in forma compatta. Il `token_id` è derivato deterministicamente come $\text{HMAC-SHA256}(K_{HMAC}, \text{identity_id} || \text{election_id})$ dove K_{HMAC} è una chiave di 32 byte casuali, **separata** dalla chiave PSS: stessa identità e stessa elezione producono sempre lo stesso `token_id`, consentendo il recupero del token dopo crash senza memorizzare alcuna mappatura. Nella demo, la custodia in HSM prescritta dal WP2 è **simulata** da un oggetto che incapsula K_{HMAC} in memoria, espone la sola operazione `hmac(identity_id, election_id)`, non offre alcun metodo di esportazione della chiave e accoda ogni invocazione a un log interno (controfigura del log firmato); in produzione il modulo va sostituito da un HSM reale con backup tramite gli share di Shamir di `shamir.py`. Il token non contiene il campo `role`: l'EA è il solo decisore sull'accettazione del ballot, tramite il Registro dei Token Spesi. Poiché RSA-PSS incorpora una *salt* casuale, ri-emettere lo stesso token produce byte di firma **diversi** pur con payload e `token_id` identici: è il `token_id` che garantisce l'unicità lato EA, non i byte del *wire*. In produzione `expires_at` deve coincidere con l'orario di chiusura delle urne; nella demo il valore è configurabile come parametro del costruttore dell'AS.

- **PBB e Merkle (pbb.py).** Catena di hash-pointer con `prev_hash` di genesi pari a 32 byte nulli. L'albero di Merkle è binario con promozione dei nodi dispari (duplicati con sé stessi); un albero vuoto produce comunque una radice (SHA-256 della stringa vuota) per garantire sempre un valore firmabile. La prova di inclusione è una lista di coppie (*sibling*, L/R).
- **Scrutinio (ea_server.py).** Il registro anti-replay è un `set` Python ($\mathcal{O}(1)$); il *freeze* blocca nuovi voti all'avvio dello scrutinio. Il rimescolamento usa Fisher–Yates con `secrets.randbelow` (non `random`). La chiave SK_{EA} ricostruita viene azzerata e cancellata dopo la decifratura. Le schede con JSON malformato, `elezione_id` errato o `scelta_lista` fuori dall'insieme ammesso sono conteggiate come nulle.
- **Client (client.py).** Il B_{packet} **non** è firmato dal client (scelta deliberata: evita il non ripudio e quindi la coercizione); l'autorizzazione è portata dal solo token firmato dall'AS. Ogni scheda include un `salt_nonce` fresco di 16 byte contro attacchi a dizionario sul ridotto spazio dei testi in chiaro.
- **Tradeoff di sicurezza della derivazione HMAC.** La derivazione deterministica del `token_id` ha un duplice effetto, opposto. Da un lato **elimina il vettore di doppio voto** introdotto dal modello a re-emissione casuale: ri-autenticarsi restituisce lo stesso `token_id`, che se già speso viene rifiutato dall'EA. Dall'altro **introdurrebbe, se la chiave fosse esposta, la linkabilità identità↔scheda**: il detentore di K_{HMAC} può ricalcolare il `token_id` di qualsiasi elettore e confrontarlo con i `token_id` pubblicati nella PBB, risalendo al ciphertext di quell'elettore; in collusione con l'EA, che detiene SK_{EA} e può decifrare, si ricostruirebbe il legame identità→preferenza. Questa valutazione coincide con quella della Sezione 3.2.4: il vettore è qualitativamente più forte del design a token casuale, e per questo **la custodia di K_{HMAC} in HSM con backup a soglia non è un'estensione futura ma un requisito del protocollo** (Sezione 2.2), che la demo simula con il modulo descritto sopra. *Frase di sintesi: la derivazione HMAC scambia robustezza contro il doppio voto con una concentrazione di rischio su K_{HMAC} ; il protocollo è sicuro nella misura in cui quella chiave gode delle stesse garanzie di custodia di SK_{EA} .*

4.5 Prestazioni Sperimentali

Le misure (Tabella 3) sono state ottenute su Python 3.14.2 con `cryptography` 46.0.5, come media di 10 esecuzioni tramite `time.perf_counter`. I tempi assoluti dipendono dalla macchina, mentre le dimensioni dei messaggi sono deterministiche e riproducibili. Le tabelle del presente documento riportano le medie, mentre l'output completo (media $\pm$ deviazione) è riproducibile eseguendo la suite dal repository, insieme alla specifica della macchina di prova stampata in testa al report.

Operazione	Misura
Generazione chiavi RSA-4096	302,08 ms (una tantum, allo startup)
Cifratura OAEP (1 scheda)	0,07 ms; ciphertext 512 byte
Firma PSS (1 token/foglia)	3,19 ms; firma 512 byte
Verifica PSS (1 token/foglia)	0,06 ms
Dimensione totale B_{packet}	1947 byte
Dimensione ricevuta	1125 byte
Costruzione Merkle ($N = 10/100/1000$)	0,010 / 0,086 / 0,593 ms
Prova di inclusione Merkle ($N = 1000$)	0,0016 ms; 320 byte (10 nodi)
Flusso di voto completo (1 votante)	~3,2 ms
Lookup anti-replay	0,0795 μ s ($\mathcal{O}(1)$)

Tabella 3: Risultati dei benchmark (media di 50 esecuzioni con warm-up esplicito).

Lettura dei risultati. Il costo dominante è la generazione delle chiavi RSA-4096 (~ 302 ms), che è tuttavia un costo *una tantum* sostenuto allo startup di AS ed EA e non grava sul singolo voto. Il flusso per-votante ($\sim 3,2$ ms) misura il solo percorso autenticazione–cifratura–invio–ricevuta a chiavi già generate, ed è dominato dalla firma PSS della ricevuta ($\sim 3,19$ ms): la cifratura OAEP della scheda, per contro, è praticamente istantanea (0,07 ms). Ciphertext e firma occupano 512 byte ciascuno, coerentemente con un modulo RSA-4096; la codifica esadecimale ne raddoppia l’ingombro nel B_{packet} trasmesso. La costruzione dell’albero di Merkle scala linearmente nel numero di schede, mentre la prova di inclusione è logaritmica (10 nodi, 320 byte, per $N = 1000$). Infine, il lookup anti-replay su **set** ($\sim 0,08 \mu s$, indipendente dal numero di token spesi) conferma empiricamente la complessità $\mathcal{O}(1)$ dichiarata nel WP2.

Complessivamente, i risultati confermano il rispetto del **vincolo di efficienza** posto fin dal WP1: il carico per-votante è di pochi millisecondi e domina sul client la sola cifratura asimmetrica leggera, mentre le operazioni costose (generazione chiavi, decifratura massiva) restano confinate al server o alla fase asincrona di scrutinio.

Prestazioni al variare del numero di votanti. La Tabella 4 riporta i risultati dello stesso flusso eseguito a scala crescente, con warm-up esplicito e numero di ripetizioni adattivo ($N = 100$: 50 ripetizioni; $N = 1.000$: 20 ripetizioni; $N = 10.000$: 5 ripetizioni). Il keygen è escluso (eseguito una sola volta all’avvio); i tempi di token generation, OAEP e PBB insert sono medie per votante; merkle build, scrutinio e flusso totale sono costi aggregati dell’intera elezione.

N	T.gen (ms/vot.)	Enc (ms/vot.)	PBB (ms/vot.)	Merkle build (ms)	Proof (ms)	Scrutinio (ms tot.)	Flusso tot. (ms)
100	3,25	0,07	0,002	0,086	0,001	503,2	1.162,6
1.000	3,19	0,08	0,002	0,593	0,002	3.253,1	9.771,3
10.000	3,17	0,08	0,002	6,173	0,002	30.693,0	95.618,0

Tabella 4: Benchmark di scaling (keygen escluso, warm-up esplicito). T.gen e Enc sono medie per votante; Merkle build, Scrutinio e Flusso tot. sono costi aggregati.

Lettura dei risultati di scaling. La cifratura OAEP (Enc) e l’inserimento in PBB risultano costanti al variare di N ($\sim 0,07$ – $0,08$ ms e $\sim 0,002$ ms per votante rispettivamente), confermando la natura $\mathcal{O}(1)$ di queste operazioni. La firma PSS (T.gen) mostra ora, dopo l’introduzione del warm-up esplicito, un andamento stabile e coerente tra le tre scale: 3,25, 3,19 e 3,17 ms, variazione inferiore al 3%, compatibile con la sola fluttuazione statistica residua. L’instabilità osservata nelle run precedenti era quindi interamente attribuibile all’inizializzazione lazy di OpenSSL alla prima invocazione, non a un degrado strutturale della pipeline. Il Merkle build cresce **linearmente** ($\times \sim 10$ a $N \times 10$), confermando la complessità $\mathcal{O}(N)$ della costruzione, mentre la prova di inclusione scala con $\mathcal{O}(\log N)$ come atteso. Lo scrutinio ($\sim 3,1$ – $3,2$ ms/scheda) è il termine dominante e scala linearmente: a $N = 10.000$ richiede ~ 31 secondi. Va precisato che il flusso totale è un costo **aggregato seriale**: il benchmark elabora i votanti in sequenza su una singola macchina, per cui il tempo totale cresce con N per costruzione e non è rappresentativo della latenza reale di un sistema distribuito con richieste concorrenti. Per l’elezione del Consiglio degli Studenti UNISA, con un bacino realistico di $N < 2.000$ votanti, l’intero scrutinio si conclude in meno di 7 secondi. **Conformità con il WP2.** Il prototipo implementa fedelmente le sei fasi del protocollo descritto nel WP2. Le deviazioni e gli aggiornamenti della revisione v3 sono stati propagati in entrambe le direzioni: la pubblicazione del token anonimo firmato in ciascuna entrata della PBB è recepita nelle Sezioni 2.4 e 3.3; la custodia di K_{HMAC} prescritta dalla Sezione 2.2 è simulata dal modulo HSM descritto nella Sezione 4.4; la verifica del fingerprint di PK_{EA} (Sezione 2.6) è realizzata in `client.py` come confronto fra l’hash SHA-256 della chiave scaricata e la prima entrata della PBB.